

Trabalho Prático
Análise de Débitos Diretos (PS2)

Grupo 99 — Licínio Cunha (nº 25454)

Docente: Óscar Ribeiro

Laboratórios de Informática — LESIPL — 2025/26

Índice

1	Introdução	2
2	Formato PS2 e Leitura dos Dados	2
3	Validação	3
4	Dashboard	4
5	Testes	5
6	Organização do Projeto	6
7	Conclusão	7

Lista de Figuras

4.1	Vista principal do dashboard, com os filtros de período/cliente e a tabela de Alertas de Validação	5
6.1	Fluxo de dados: do ficheiro PS2 ao dashboard	6
6.2	Hierarquia de pastas do projeto	7

Lista de Tabelas

2.1	Posições dos campos no formato PS2 simplificado	2
3.1	Resultados da validação dos ficheiros de <code>data/</code>	4
5.1	Distribuição dos testes automáticos	6

Listings

2.1	Listagem de ficheiros PS2 (<code>ps2/leitura.py</code>)	3
2.2	Parsing de um registo de movimento (<code>ps2/parsing.py</code>)	3
3.1	Validação de NIF (<code>ps2/validacao.py</code>)	3
4.1	Agregação da funcionalidade própria (<code>ps2/agregacao.py</code>)	4
5.1	Carregamento de alto nível (<code>ps2/carregador.py</code>)	6

1. Introdução

Os *débitos diretos* são um dos meios de cobrança mais usados entre empresas e bancos em Portugal, comunicados através de ficheiros num formato de largura fixa conhecido por **PS2**. Este trabalho tem como objetivo ler, validar e analisar um conjunto de ficheiros PS2 (versão simplificada) com os débitos diretos recebidos por uma instituição bancária fictícia, *IPCA Energy*, e apresentar os resultados num *dashboard* interativo.

O trabalho foi desenvolvido individualmente, seguindo o paradigma funcional (declarativo) pedido no enunciado: as funções que leem, validam e agregam os dados são *puras* (sem efeitos secundários), recorrendo a *dataclasses* imutáveis, compreensões de listas/dicionários e funções de ordem superior (**map**, **reduce**) em vez de ciclos com alteração de estado.

Este documento organiza-se da seguinte forma: o Capítulo 2 descreve o formato PS2 e a estratégia de leitura; o Capítulo 3 descreve as regras de validação aplicadas (estrutura, NIF e NIB); o Capítulo 4 apresenta o *dashboard* desenvolvido, incluindo a funcionalidade própria; o Capítulo 5 descreve a estratégia de testes automáticos; o Capítulo 6 descreve a organização do repositório; e o Capítulo 7 fecha com as conclusões e o trabalho futuro.

2. Formato PS2 e Leitura dos Dados

Cada ficheiro DD_AAAA_MM.ps2 é composto por linhas de largura fixa, com três tipos de registo:

Registo de Cabeçalho (tipo 1) primeira linha do ficheiro; contém a data de processamento, o nome e NIF da empresa ordenante, e os totais declarados (valor e número de operações).

Registo de Movimento (tipo 2) uma linha por cada débito a um cliente, identificado pelo respetivo NIB.

Registo de Fim (tipo 9) última linha; repete os totais do cabeçalho para efeitos de validação e controlo.

As posições de cada campo (Tabela 2.1) foram confirmadas por inspeção direta dos ficheiros de exemplo fornecidos, uma vez que descrevem uma versão simplificada do *layout* PS2 oficial.

Registo	Campo	Posição (base 1)
Cabeçalho (1)	data (AAAAMMDD)	2–9
	empresa	10–52
	NIF	53–61
	total (cêntimos)	62–75
	nº de registos	76–81
Movimento (2)	referência / ADC	2–11
	NIB (21 dígitos)	12–32
	importância (cêntimos)	42–55
	descrição	56–82
Fim (9)	total (cêntimos)	2–15
	nº de registos	16–21

Tabela 2.1: Posições dos campos no formato PS2 simplificado

A listagem de ficheiros usa `pathlib`[2] para descobrir, de forma ordenada, todos os ficheiros DD_*.ps2 numa pasta, sem depender de nomes escritos manualmente (Listagem 2.1):

Listing 2.1: Listagem de ficheiros PS2 (ps2/leitura.py)

```
def listar_ficheiros_ps2(pasta: str) -> list[Path]:
    return sorted(Path(pasta).glob("DD_*.ps2"))
```

Cada linha é depois interpretada por *slicing* em objetos de domínio imutáveis (RegistoInicio, RegistoMovimento, RegistoFim), como no exemplo da Listagem 2.2 para os registos de movimento:

Listing 2.2: Parsing de um registo de movimento (ps2/parsing.py)

```
def parse_movimento(linha: str) -> RegistoMovimento:
    return RegistoMovimento(
        referencia=linha[1:11],
        nib=linha[11:32],
        importancia_cent=int(linha[41:55]),
        descricao=linha[55:].strip(),
    )
```

3. Validação

A validação está organizada em três níveis, todos implementados em ps2/validacao.py através de funções puras que devolvem bool ou listas de mensagens de erro:

1. **Estrutura do ficheiro:** a primeira linha tem de ser tipo 1, a última tipo 9, todas as intermédias tipo 2; a data de processamento tem de ser válida; e o total/número de registos declarados no cabeçalho e no fim têm de coincidir com a soma real dos movimentos.
2. **NIF do ordenante** (Anexo A): 9 dígitos, primeiro dígito em {1, 2, 5, 6, 8, 9} e dígito de controlo calculado por soma ponderada módulo 11.
3. **NIB do cliente** (Anexo B): 21 dígitos e resto 1 na divisão por 97 do número completo, tratado como inteiro de precisão arbitrária (nativo em Python, ao contrário de outras linguagens onde excede os tipos inteiros nativos).

A validação do NIF (Listagem 3.1) segue diretamente o Anexo A: o dígito de controlo é calculado por soma ponderada dos primeiros 8 dígitos e comparado com o 9º.

Listing 3.1: Validação de NIF (ps2/validacao.py)

```
def valida_nif(nif: str) -> bool:
    if len(nif) != 9 or not nif.isdigit():
        return False
    if nif[0] not in "125689":
        return False
    soma = sum(int(nif[i]) * (9 - i) for i in range(8))
    resto = soma % 11
    controlo = 0 if resto in (0, 1) else 11 - resto
    return controlo == int(nif[8])
```

Nota importante: ao implementar literalmente o algoritmo do Anexo B, verificou-se que o *próprio exemplo do enunciado* (003506510000258741254) dá resto 96, não 1 — ou seja, não verifica a regra que o enunciado apresenta como exemplo de NIB válido. O mesmo acontece com o NIF de exemplo do Anexo A. Optou-se por implementar as regras exatamente como descritas (com testes automáticos que documentam esta discrepância) e assinalar a questão para confirmação com o docente, em vez de “corrigir” silenciosamente o algoritmo.

Aplicando estas regras aos 35 ficheiros em `data/` obtém-se o resumo da Tabela 3.1: a estrutura está sempre correta (totais e contagens coerentes), mas o NIF do ordenante e todos os NIBs de cliente falham a validação literal — o que motivou a funcionalidade própria descrita no capítulo seguinte.

Verificação	Ficheiros afetados	Total	%
Estrutura (totais/contagens)	0 / 35	35	0%
NIF do ordenante inválido	35 / 35	35	100%
NIBs de cliente inválidos	350 / 350	350	100%

Tabela 3.1: Resultados da validação dos ficheiros de `data/`

4. Dashboard

O *dashboard* foi implementado com a *framework* Shiny for Python[1] (`src/app.py`) e disponibiliza três grupos de vistas:

- **Evolução mensal:** gráfico de linha do total faturado, filtrável por mês inicial e final.
- **Comparação de clientes:** gráfico de barras com o total por cliente (NIB) no período completo, e um segundo gráfico com a evolução mensal de um cliente selecionado.
- **Alertas de Validação** — *funcionalidade própria*: uma tabela que resume, ficheiro a ficheiro, o NIF do ordenante, se é válido, e quantos NIBs de cliente falham a validação, acompanhada de um contador global de NIBs inválidos. Esta vista aproveita diretamente as funções de `ps2/validacao.py`, agregadas em `ps2/agregacao.py` através de uma compreensão de lista (Listagem 4.1), e dá visibilidade imediata a problemas de qualidade de dados sem ser preciso consultar a consola.

Listing 4.1: Agregação da funcionalidade própria (`ps2/agregacao.py`)

```
def resumo_alertas(ficheiros: list[Ficheiro]) -> list[dict]:
    return [
        {
            "periodo": periodo(f),
            "nif": f.inicio.nif,
            "nif_valido": valida_nif(f.inicio.nif),
            "n_movimentos": len(f.movimentos),
            "n_nibs_invalidos": sum(
                1 for m in f.movimentos if not valida_nib(m.nib)
            ),
        }
        for f in ficheiros
    ]
```

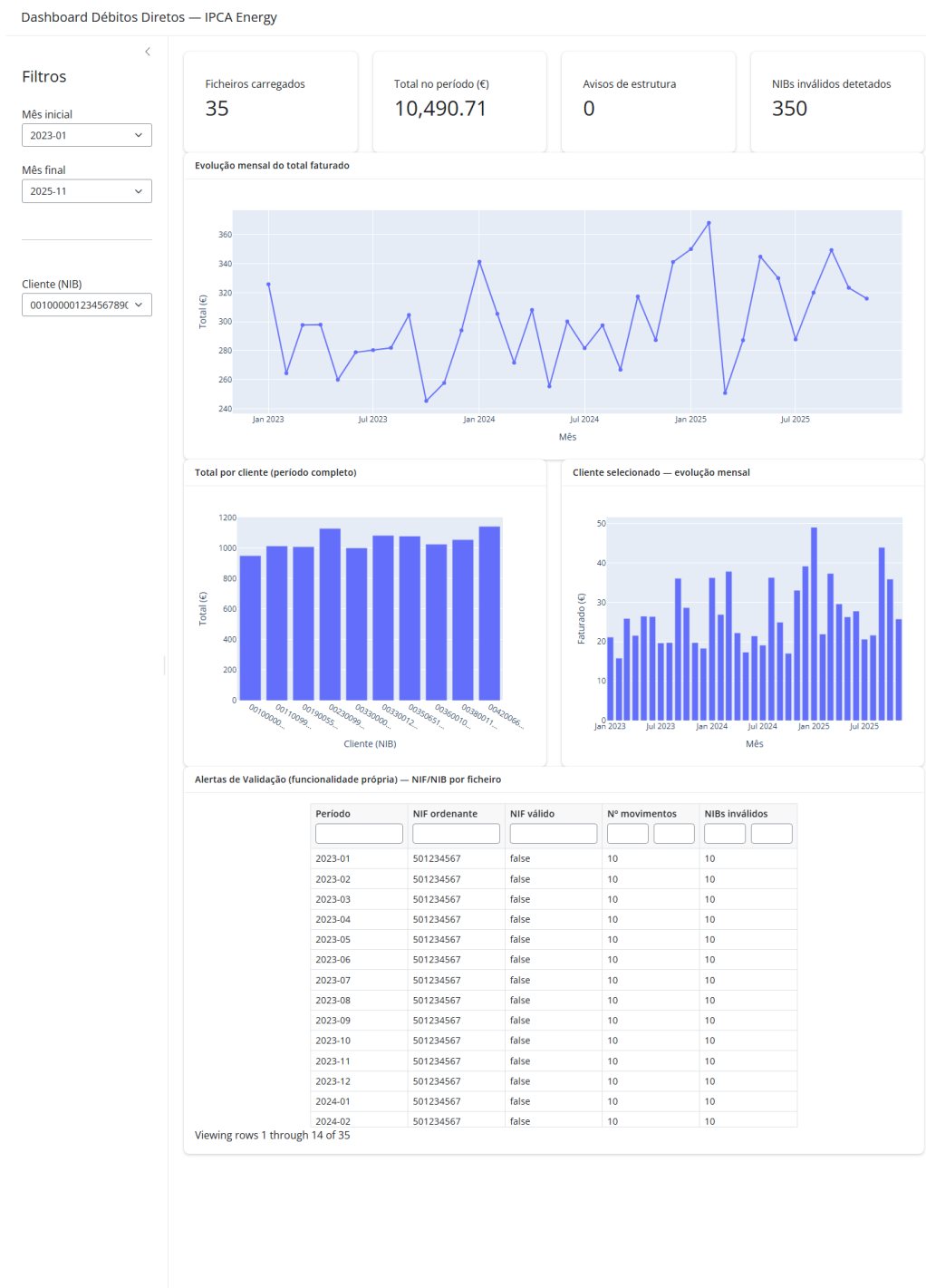


Figura 4.1: Vista principal do dashboard, com os filtros de período/cliente e a tabela de Alertas de Validação

5. Testes

Seguiu-se uma abordagem próxima de TDD: cada módulo do pacote **ps2** tem um ficheiro de testes correspondente em **tests/**, usando **pytest**. Os testes dividem-se em dois níveis (Tabela 5.1): testes unitários, que isolam uma função pura (ex.: um NIF ou NIB específico, uma linha de exemplo), e um teste de integração que carrega a pasta **data/** completa e confirma que os 35

ficheiros ficam estruturalmente válidos.

Ficheiro	Nº testes	Cobre
test_validacao.py	11	estrutura, NIF, NIB
test_parsing.py	3	<i>parsing</i> de cabeçalho/movimento/fim
test_agregacao.py	6	totais por mês/cliente, alertas
test_carregador.py	2	integração com <i>data</i> /
Total	22	

Tabela 5.1: Distribuição dos testes automáticos

O teste de integração (Listagem 5.1) exercita `carregar_pasta`, que combina listagem, leitura e validação de estrutura numa única função de alto nível, devolvendo separadamente os ficheiros válidos e os avisos encontrados:

Listing 5.1: Carregamento de alto nível (`ps2/carregador.py`)

```
def carregar_pasta(pasta: str = "./data") -> tuple[list[Ficheiro], list[
    str]]:
    ficheiros: list[Ficheiro] = []
    avisos: list[str] = []
    for caminho in listar_ficheiros_ps2(pasta):
        erros = valida_estrutura(caminho.name, ler_linhas(caminho))
        if erros:
            avisos.extend(erros)
        else:
            ficheiros.append(carregar_ficheiro(caminho))
    return ficheiros, avisos
```

Os testes correm com `pytest -v` (ou `make test`, ver `Makefile`) e estão também configurados no `.vscode/settings.json` para deteção automática no VS Code.

6. Organização do Projeto

O código está modularizado em ficheiros de responsabilidade única dentro do pacote `ps2` (Figura 6.2): leitura, modelo de domínio, *parsing*, validação, agregação e carregamento de alto nível, cada um testado isoladamente em `tests/`. Esta separação permite testar, por exemplo, a validação de NIF sem depender da leitura de ficheiros reais.

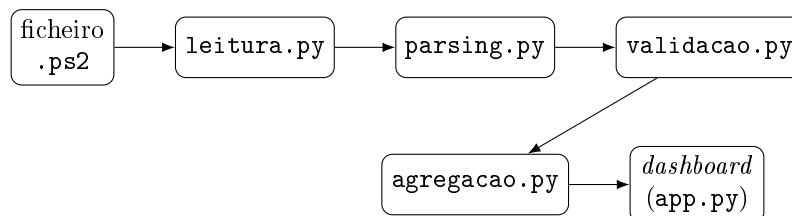


Figura 6.1: Fluxo de dados: do ficheiro PS2 ao dashboard

O repositório Git segue uma branch por funcionalidade (`feature-*`), integrada em `main` através de *pull requests* no GitHub, sem eliminar as branches após a integração — estratégia

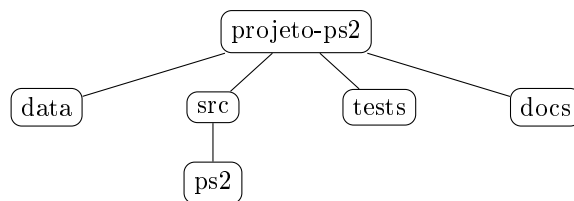


Figura 6.2: Hierarquia de pastas do projeto

detalhada no `README.md`.

7. Conclusão

O trabalho cumpre o ciclo pedido no enunciado: leitura dos ficheiros PS2, validação de estrutura e de NIF/NIB, agregação da informação e um *dashboard* interativo com uma funcionalidade própria de alertas de validação. A maior dificuldade foi perceber que os próprios exemplos dos Anexos A e B do enunciado não verificam o algoritmo que descrevem; optou-se por implementar as regras tal como documentadas e documentar a discrepância em testes automáticos, em vez de adaptar silenciosamente o critério de validação.

Como trabalho futuro, ficam por explorar: exportação dos alertas de validação para CSV, deteção de meses atípicos por desvio-padrão em relação à média histórica, e suporte ao IBAN (PT50 + NIB), que está a substituir o NIB isolado nas comunicações bancárias.

Bibliografia

- [1] Posit Software. Shiny for python. <https://shiny.posit.co/py/>. Consultado em dezembro de 2025.
- [2] Python Software Foundation. The python standard library — pathlib. <https://docs.python.org/3/library/pathlib.html>. Consultado em dezembro de 2025.